

第 1 章 操作系统基础

1.1 操作系统四大功能；四大特征（重点）， 每个的含义

四大功能：**进程管理：**保证 *CPU* 正确地同时运行多道程序。主要任务是处理进程的建立、控制、通信、同步以及系统资源的共享。**存储管理：**主要任务是管理内存资源。包括为程序分配内存、保护用户程序和数据不被破坏，以及实现虚拟存储器以逻辑上扩充内存容量。**设备管理：**管理输入输出 (*I/O*) 设备，使多个用户能共享和方便地使用设备。包括设备分配、启动和控制设备、处理中断、管理缓冲区等。**文件管理：**对计算机的程序、数据等信息资源进行组织和管理，向用户提供按名称进行存储、检索、查询、保护等的接口。*现代操作系统还扩展了网络通信与服务、安全与保护功能。*

四大特征：**并发性：**两个或多个事件在同一时间间隔内发生的、同时处于活动状态的特性。微观上程序是交替运行的（串行），宏观上都在运行（并发）。

共享性：内存中并发运行的多个程序可以共享计算机的硬件和软件资源。通常分为互斥共享（如打印机，某一时刻只能一个进程用）和交替共享（如磁盘访问）。

虚拟性：将一个物理实体映射为一个或多个逻辑对象。比如通过多道程序设计和分时使用，把一个物理 *CPU* 虚拟为多个逻辑 *CPU*。**随机性（异步性）：**每道程序在何时运行、各个程序运行的顺序，以及每道程序所需的时间都是不确定的、不可预知的。

1.2 联机批处理和脱机批处理，多道；并发并行。操作系统的分类，区别

联机批处理：监督程序将成批作业从输入设备逐个输入磁带，再逐个装入内存编译运行。此时主机与外设依然是串行工作。**脱机批处理：**增加了一台不与主机直接相连的“卫星机”。成批作业通过卫星机输入磁带供主机调用，运行结果输出到磁带再由卫星机打印。使主机与外设变成了并行工作，提高了效率。

多道：多道程序设计的基本思想是在内存中同时存放几道程序，它们交替运行（如一个等待 *I/O* 时，*CPU* 去执行另一个），以提高 *CPU* 和资源利用率。

并发与并行：并行是指多个事件在同一时刻发生（依赖多 *CPU*）；并发是指多个事件在同一时期内发生（单 *CPU* 下微观交替、宏观同时）。并行是并发的特例。

操作系统的分类与区别：

批处理操作系统：用户将作业提交给系统，系统自动顺序运行。适应已调试好、不需要用户交互的大作业。目标是提高资源利用率和系统吞吐量。**分时操作系统：**通过终端连接，主机把 *CPU* 时间划分为小“时间

片”轮流分给用户。适应需要交互的作业。具有同时性、交互性、独立性、及时性的特征，目标是满足用户的交互和及时响应。**实时操作系统**：计算机必须及时响应外部事件的请求，并以足够快的速度完成处理。用于过程控制（如导弹、工业控制）和实时信息处理。其对响应速度的要求比分时系统高得多。

1.3 程序接口，操作接口；系统调用的作用、优点，和普通函数调用的区别

程序接口：操作系统提供给程序使用操作系统服务和功能的接口，通常通过**系统调用**和 **API**（应用程序编程接口）来实现。

操作接口：操作系统向用户提供的操控计算机的接口，包括命令界面（**CUI/CLI**）、图形界面（**GUI**）和作业控制命令语言。

系统调用的作用和优点：是为了扩充机器功能、增强系统能力、方便用户使用而建立的。用户程序通过它可以访问系统底层软硬件资源，而不必了解系统内部结构和硬件细节，使用安全且方便。

与普通函数调用的区别：普通函数调用通常在用户环境（用户态）下完成，且不能脱离环境运行；系统调用（广义指令）是由操作系统在机器指令基础上实现的过程，调用时通常会引起处理器模式的切换（从用户态切换到核心态）。

第 2 章 进程管理

2.2 进程，程序，线程，作业（概念）；并发，并行

进程：是可并发运行的程序在某个数据集合上的一次运行过程，是操作系统资源分配、保护和调度的基本单位。具有动态性。**程序**：是代码文件，强调其静态性和持久性，包含机器指令或高级语言。**线程**：是操作系统进程中能够独立执行的实体（控制流），是处理器调度和分派的基本单位。**作业**：一般指批处理系统要装入系统运行处理的一系列程序和数据。

2.3 进程状态转换——三态转换及原因；

三态转换及原因：**就绪** → **运行**：当 **CPU** 空闲时，操作系统从就绪队列中选中该进程并分配 **CPU**。**运行**

→ **阻塞**：正在运行的进程由于需要等待某些事件（如请求 **I/O** 操作）的发生而主动放弃 **CPU**。**阻塞** →

就绪：进程等待的事件到来（如 **I/O** 完成），不需要再等待时，被唤醒进入就绪队列。**运行** → **就绪** 是因为时间片用完或被高优先级进程抢占。

PCB 的作用：进程控制块（**PCB**）记录了描述进程当前状态以及控制进程运行的信息（标识信息、现场信息、控制信息）。系统通过 **PCB** 来管理和控制进程，**PCB** 是**进程存在的唯一标志**。

核心态和用户态：指CPU的运行模式。核心态（管态）是系统内核和驱动运行的模式，拥有无限制的指令执行特权，可自由访问所有内存和外设；用户态（目态）是一般用户进程运行的模式，受到安全限制，只能访问自身映射的虚拟地址空间。

2.4 进程控制：临界区，临界资源，控制规则

临界资源：某段时间内只能允许一个进程使用的资源（如打印机、共享变量）。

临界区：访问临界资源的那段代码。

控制规则（调度原则）：1) 一次至多一个进程进入临界区执行；2) 已有进程在临界区时，其他试图进入的进程应等待；3) 进入临界区内的进程应在有限时间内退出。

互斥：若干进程因相互竞争独占性资源而产生的竞争制约关系。

同步：并发进程为完成共同任务，基于某个条件协调其运行进度、执行次序而产生的协作制约关系（互斥是一种特殊的同步）。

2.5 进程通信：通信方式，适用范围，特点

消息传递通信：进程将数据封装在“消息”中，通过消息缓冲区传递。分为直接通信和间接通信（邮箱）。

特点是灵活、隐藏了实现细节、简化程序设计，适用范围广。**共享内存通信：**在内存中划出一块共享存储区，多个进程通过对该区域的读写实现通信。**特点是高效，适用于进程间需传送大量数据的情况。****管道通信(Pipe)：**管道是连接读写进程的一个特殊文件，按FIFO（先进先出）方式传送数据流。**特点是需要严格同步，可跨地址空间。包含匿名管道和命名管道（支持跨网络）。**

2.6 进程调度（重点）算法；三级调度；调度算法计算

三级调度模型：**高级调度（作业调度）：**从后备作业中选出若干作业，分配资源并建立进程调入内存。**中级调度（平衡调度）：**内存紧张时将暂时不能运行的进程挂起换至外存，内存有空闲时再换入。**低级调度（进程调度）：**根据算法将CPU分派给就绪队列中的一个进程（运行频率极高）。

前5种必会调度算法：**先来先服务（FCFS）：**按进程就绪的先后顺序调度，非抢占式。对长作业有利、对短作业不利。**短作业优先（SJF）：**选估计需要CPU时间最短的作业优先运行，非抢占式。平均周转时间比FCFS短，但长作业可能会饿死。**最短剩余时间优先（SRTF）：**SJF的抢占式版本。如果新到达作业所需时间比当前正在运行作业的剩余时间短，则剥夺当前作业的CPU。**高响应比优先（HRRF）：**响应比 =

$1 + (\text{已等待时间} / \text{估计运行时间})$ 。兼顾等待时间和运行时间，非抢占式，不会发生饥饿。**优先权调度**

(**HPF**)：根据系统或用户赋予的优先权大小（静态或动态）选取最高的进程执行，通常是抢占式的。

2.7 死锁，必要条件

死锁：一组进程均只占有部分所需资源而无法继续执行，陷入永远互相等待其他进程释放资源的状态。

四个必要条件：**互斥条件**：资源的使用是互斥的。**请求与保持条件**：进程得到部分资源后请求新资源受阻时，对已得资源保持不放。**不剥夺条件**：进程获得资源后只能由其自身主动释放，不能被剥夺。**环路等待条件**：系统中若干进程形成一个相互等待资源的闭合环路。

2.8 线程的概念，线程与进程的区别与联系

线程概念：是操作系统进程中能够独立执行的控制流实体，是处理器调度和分派的基本单位。

区别与联系：**调度与拥有资源**：进程是拥有资源的基本单位，而线程是独立调度的基本单位。线程自己并不拥有大量系统资源（仅有如寄存器、栈等少部分），但可以访问和共享其隶属进程的全部资源（代码、数据、打开的文件等）。

并发性：引入线程后，不仅不同进程可以并发，同一进程内的多个线程也可以并发，并发粒度更小、吞吐量更高。

系统开销：进程的创建、撤销和切换涉及虚拟地址空间的变更，时空开销极大；而线程切换只需保存和设置少量寄存器，不涉及内存管理，切换开销显著小于进程。

根据您上传的第二个 PDF (《操作系统复习重点》) 中第 3 章和第 4 章的考点要求，我为您提取并整理了

教材中对应的核心知识点（去除页码，直接呈现内容）：

第 3 章 内存管理

3.1 覆盖与交换技术的比较

覆盖技术 (Overlay)：主要用于早期内存较小的系统。它将一个程序分成若干个不能同时执行的模块

(覆盖块)，只有程序当前需要的模块才放入内存，不需要的模块存放在外存。这要求程序员明确指定覆盖结构，主要发生在一个程序内部。**交换技术 (Swapping)**：当内存空间紧张时，操作系统自动将内存中暂时不能运行的进程（或整个程序）换出到外存，把具备运行条件的进程换入内存。交换主要发生在不同进程之间，是由操作系统透明管理的，目的是提高内存利用率和系统吞吐量。

3.2 动态可变分区分配算法 (5 种)

首次适应算法 (First Fit)：空闲分区按**地址递增**次序链接。分配时从头开始查找，找到第一个满足大小要求的空闲分区即分配。

循环首次适应算法 (Next Fit)：又称邻近适应算法。分配时从**上次找到的空闲分区的下一个分区**开始查找，找到第一个满足要求的即可。

最佳适应算法 (Best Fit)：空闲分区按**容量递减**次序链接。找到能满足要求且**最小**的空闲分区，这样保留了较大的空闲区。但缺点是容易产生很多极小且无法利用的外部碎片。

最坏适应算法 (Worst Fit)：空闲分区按**容量递增**次序链接。找到**最大**的空闲分区进行分配，克服了产生极小碎片的缺点，但会导致系统中的大空闲区被消耗殆尽。

快速适应算法 (Quick Fit)：将空闲分区按容量大小分类，并各自建立单独的链表。分配时直接到满足要求且容量最接近的类别链表中去取。查找效率最高，但合并回收的开销大。

3.3 页式存储管理：；快表的作用

快表 (TLB) 的作用：快表是一组关联的高速缓存寄存器，用来存放当前最近访问过的页表项。其作用是：在地址转换时先查快表，若命中则直接得出物理块号。这避免了访问主存中的页表，从而**大大加快了地址转换和内存访问的速度**。

3.4 段式中分段和分页的区别；段页式原理

分段和分页的区别：

目的：分页是出于**系统管理的需要**（消除碎片，提高内存利用率）；分段是出于**用户的需要**（方便编程、共享和保护）。**大小**：页的大小是**固定**的，由硬件决定；段的大小是**可变**的，取决于用户编写的程序。**逻辑地址空间**：分页的地址空间是**一维**的；分段的地址空间是**二维**的（必须分别给出段号和段内偏移量）。

段页式原理：结合了分段和分页的优点。将用户的逻辑程序先**分段**（方便管理和共享），再将每个段划分为大小相等的页。地址转换时，需要查两次表：先查段表获取该段的页表起始地址，再查页表获取对应的物理块号，最后加上页内偏移量形成物理地址。

3.6 请求式置换算法（分配/调入策略及四种常考算法）

分配与调入策略：内存分配策略包括固定分配局部置换、可变分配全局置换、可变分配局部置换；页面调入策略分为请求调页（用到才调入）和预调页（提前预判调入）。

页面置换算法：

最佳置换算法 (OPT)：淘汰以后永远不再使用或在最长时间不再被访问的页面。这是理论上的最优算法，无法实现，仅用于评价其他算法的性能。

先进先出置换算法 (FIFO)：淘汰最先进入内存的页面（即驻留时间最长的页面）。简单但性能差，易产生异常（分配的物理块增加，缺页率反而升高）。

最近最久未使用置换算法 (LRU)：淘汰在最近一段时间里最久没有被访问过的页面。基于局部性原理，性能较好，是常考重点。

时钟置换算法 (Clock/NRU)：将页面链接成一个循环队列，并为每页设置一个访问位。缺页时循环扫描，若访问位为 1 则清 0（给一次机会），若为 0 则直接淘汰该页。

第 4 章 设备管理

4.1 设备分类

按数据传输率/信息交换单位分类：分为**块设备**（如磁盘，以数据块为单位，传输率高，可寻址寻道）和**字符设备**（如键盘/打印机，以字符为单位，不可寻址）。

按资源分配共享方式分类：分为**独占设备**（分配给一个进程后直到其释放）、**共享设备**（可供多个进程交替寻址使用，如磁盘）、**虚拟设备**（通过 *SPOOLing* 技术将独占设备改造为可共享的设备）。

4.2 设备四种控制方法（原理及优缺点）

程序循环查询方式：*CPU* 不断轮询检查设备状态。**优点是**硬件结构简单；**缺点是***CPU* 与外设完全串行工作，*CPU* 绝大多数时间处于盲等，资源极大浪费。

中断驱动方式：*I/O* 操作完成时，外设向 *CPU* 发送中断信号。**优点是**实现了 *CPU* 与外设的部分并行工作，提高了 *CPU* 利用率；**缺点是**每次小单位数据（如字节）传输都要中断 *CPU*，大量数据传输时仍然消耗较多 *CPU* 时间。

直接内存访问（DMA）方式：增加了 DMA 控制器，数据成块地直接在内存和外设之间传输，仅在传送开始和结束时才需 CPU 干预。**优点是大大减轻了 CPU 负担；缺点是 DMA 控制器的灵活性较差。**

通道控制方式：引入 I/O 通道（专用的 I/O 处理机），通道能执行自己的 I/O 指令独立控制总线。**优点是实现了 CPU、通道、外设的高度并行，CPU 只需下达一条指令即可完成整个 I/O 任务；缺点是系统结构复杂，硬件成本高。**

4.3 缓冲技术分类

单缓冲：在内存设一个缓冲区，设备向缓冲区输入数据，满后 CPU 再取出处理，处理和输入不能完全并行。**双缓冲：**设置两个缓冲区，设备向缓冲区 1 输入数据时，CPU 可从缓冲区 2 取出数据处理，极大提高了并发度。**多缓冲（循环缓冲）：**多个缓冲区链接成循环队列（包含空缓冲、已满缓冲等），适应读写速度差异更大的情况。**缓冲池：**统一管理多个缓冲区块，划分为空缓冲队列、输入队列、输出队列等，供全系统所有进程共享使用。

4.4 / 4.5 SPOOLing 原理与结构；设备独立性优点

SPOOLing（假脱机技术）原理与结构：

原理：利用多道程序设计技术和高速大容量磁盘，在主机控制下用软件模拟脱机 I/O 中的卫星机操作，将低速的独占设备改造成多进程共享的逻辑/虚拟设备。

结构：包含磁盘上的**输入井和输出井**、内存中的**输入缓冲区和输出缓冲区**，以及模拟硬件的**输入进程和输出进程**。

设备独立性的优点：

应用程序独立于具体使用的物理设备，程序中只使用“逻辑设备名”。

优点：1）方便用户编程（无需关注具体硬件细节）；2）提高设备分配的灵活性（某台物理设备损坏时，系统可重定向到另一台同类设备，程序无需修改）；3）改善系统的可适应性和扩展性（易于硬件升级）。

根据您上传的《操作系统复习重点》第五章的要求，结合教材《计算机操作系统》的体系结构和知识框架，为您整理了第五章（文件系统）的核心考点内容：

第 5 章 文件系统

5.1 文件逻辑结构和物理结构及其优缺点

逻辑结构：从用户观点出发看到的文件组织形式。

无结构文件（流式文件）：文件内部的数据是一串连续的字符流，没有明显的结构（如源代码文件、可执行文件）。

有结构文件（记录式文件）：文件由若干个记录组成，分为顺序文件、索引文件、索引顺序文件等（如数据库文件）。

物理结构（重要）：文件在外存（磁盘）上的实际存储组织形式。

连续分配（顺序结构）：将文件分配在连续的物理块中。**优点**：顺序访问速度极快；支持直接访问和随机访问。**缺点**：易产生外部碎片，磁盘空间利用率低；文件难以动态增长。

链接分配（链接结构）：文件存放在离散的物理块中，通过指针将各块链接起来。分为隐式链接（块内含指针）和显式链接（将指针集中存放在 *FAT* 文件分配表中）。**优点**：消除了外部碎片，显著提高了磁盘利用率；文件可以动态增长。**点**：只适合顺序访问，不支持随机访问（需从头遍历）；隐式链接的指针占用存储空间且可靠性差。

索引分配（索引结构）：为每个文件分配一个专门的“索引块”，其中记录了该文件所有数据块的物理地址。**优点**：既支持直接/随机访问，又允许文件动态增长，且无外部碎片。**缺点**：索引块本身会增加额外的存储空间开销（即使是很小的文件也要单独分配索引块）。

5.2 文件存储空间管理：磁盘空间分配，位示图

位示图法（重点）：利用内存中的一段连续二进制位（*Bit*）来表示磁盘中物理块的使用情况。例如，**1** 表示该块已分配，**0** 表示该块空闲。

分配过程：顺序扫描位示图，找到值为 **0** 的位，将其改为 **1**，并根据该位的行号和列号计算出对应的物理块号分配给文件。

回收过程：根据回收块的物理块号，计算出在位示图中的行号和列号，将其对应位的值从 **1** 修改为 **0**。

优点：位示图非常小，通常可以常驻内存，因此分配和回收物理块的速度极快，且只需进行简单的位运算。

5.3 多级索引与混合索引涉及的计算

多级索引：当文件非常大，一个索引块装不下所有的物理块号时，将索引块按层级组织（如二级索引、三级索引）。顶级索引块不指向数据块，而是指向次级索引块。

混合索引：将直接地址和多级间接地址结合在一个文件索引节点中（如 *UNIX* 系统的 *inode*）。

5.4 文件目录：索引节点（*i-node*），原因、优缺点

文件控制块（*FCB*）：传统目录中，每个目录项就是一个 *FCB*，包含了文件名、物理地址及文件的所有属性（大小、时间、权限等）。

索引节点（*i-node*）的引入原因：在检索目录时，系统主要是通过比对“文件名”来查找。如果 *FCB* 很大，一个磁盘块能存放的目录项就很少，导致查找文件时需要多次进行磁盘 *I/O*，效率极低。

概念：将文件名和文件的其他描述信息分开，除了文件名之外的所有信息（物理地址、属性等）提取出来，形成一个独立的数据结构，称为**索引节点**。此时，目录项只保留“文件名”和“指向该文件索引节点的指针”。

优点：极大缩小了目录项的尺寸，使得一个物理块可以存放更多的目录项，从而大幅度减少了目录检索时的磁盘 *I/O* 次数，提高了检索速度。

缺点：在找到文件名后，需要顺着指针额外进行一次磁盘访问来读取索引节点（尽管很多时候活跃的 *inode* 会被缓存在内存中）。

5.5 文件共享（静态共享，动态共享）

文件共享是指多个用户（或进程）共同使用同一个文件。

静态共享：这种共享关系在系统重启或进程结束后依然存在，主要体现在目录树的链接上。

基于索引节点（硬链接）：多个不同目录下的目录项的指针，指向同一个文件的索引节点（*i-node*）。索引节点中设有一个“链接计数器（*count*）”。删除一个目录项只是让 *count* 减 1，只有当 *count* 为 0 时，系统才真正删除该文件并释放物理块。

基于符号链接（软链接，类似 *Windows* 的快捷方式）：创建一个新的特殊文件（*Link* 类型），该文件的数据块中只包含被链接文件的“路径名”。访问时，系统根据路径名去寻找源文件。优点是可以跨文件系统甚至跨网络共享；缺点是每次访问都需要逐级检索目录，时间开销大。

动态共享：只在进程运行期间存在的共享关系。通过操作系统维护的“系统打开文件表”和“进程打开文件表”来实现。多个并发进程可能通过不同的系统调用同时打开同一个文件，此时操作系统负责管理它们的读写指针状态，并根据文件的互斥/同步机制（如文件锁）来控制并发访问。